



LYRA RECORDER

Technical Datasheet · Version 1.0

Overview

Lyra Recorder is a professional audio capture application for Android, featuring per-session calibration, hardware-style metering, hard-clip limiting, and adaptive silence detection. The signal path is implemented in floating-point Kotlin code on top of AudioRecord and MediaCodec for low-latency capture and high-quality encoding.

Audio engine

Capture API	Android AudioRecord (Kotlin Native)
Encoding API	MediaCodec + MediaMuxer (AAC)
Sample rate	44100 Hz
Bit depth	16-bit PCM (linear)
Channels	Mono
Internal processing	32-bit floating point
Buffer size	AudioRecord.getMinBufferSize (≥ 4096 samples)

Signal chain

Processing order, from microphone input to file output:

1. Conversion	PCM 16-bit $\rightarrow$ float $[-1.0, +1.0]$
2. HPF (optional)	IIR 1-pole, $\alpha \approx 0.989$ (~ 80 Hz cutoff)
3. Detection HPF	Always active, used for AUTO PAUSE only
4. RMS (detect)	For silence/voice threshold comparison
5. AUTO PAUSE	Sample counter against pause/resume thresholds
6. GAIN	User-controlled multiplier (knob)
7. LIMITER (optional)	Hard clip at ± 0.90 (~ -0.9 dBFS ceiling)
8. Quantization	Float $\rightarrow$ 16-bit PCM
9. Output	WAV writer / AAC encoder + monitor buffer
10. Analysis	FFT bands + waveform + VU RMS from processed signal

Per-session calibration

Two-phase test measures the microphone and the environment before each calibrated recording. From those measurements the app derives the meter gain, the auto-pause thresholds, and the viability of AUTO PAUSE.

Phase 1 duration	5 seconds (silence)
Phase 2 duration	5 seconds (voice)
Output: noiseFloor	Average RMS during silence phase
Output: voiceLevel	Average RMS during voice phase
Derived: meterGain	$(0.5 / (\text{voiceLevel} \times 3.0))$, clamped $[0.1, 4.0]$
Derived: pauseThreshold	$(\text{noiseFloor} \times 1.8)$, clamped $[0.002, 0.5]$

Derived: resumeThreshold	$(\text{noiseFloor} \times 3.5), \text{ clamped } [0.004, 0.8]$
Auto-pause viability	$\text{voiceLevel} \geq \text{noiseFloor} \times 4 \text{ AND } \text{noiseFloor} < 0.05$
Voice detection floor	$\text{voiceLevel} \geq 0.015 \text{ AND } \text{voiceLevel} \geq \text{noiseFloor} \times 2.0$

Metering

VU range	-20 dB to +3 dB
VU response	Asymmetric (fast attack 0.55, slow release 0.10)
VU ceiling (limiter off)	+3 dBFS
VU ceiling (limiter on)	-1 dBFS (hard, two-stage clamp)
Clip LED threshold	≥ -3 dBFS red, ≥ -7 dBFS amber
Spectrum bands	64 (time-domain RMS segments)
Waveform resolution	120 samples / frame
Refresh rate	~60 Hz (Flutter Ticker)

Output formats

WAV	PCM 16-bit, 44.1 kHz, mono, uncompressed
AAC	MPEG-4 AAC-LC, 256 kbit/s, 44.1 kHz, mono, .m4a container
File naming	lyra_<timestamp_ms>.<ext>
Markers file	Same name, .json sidecar

Features

Recording modes	Direct (raw) · Calibrated (full functions)
Calibration gate	HPF, LIMITER, AUTO PAUSE require active profile
Markers	Per-recording, with optional text note (≤ 50 chars)
Auto-pause modes	OFF · 1 s silence · 2 s silence
Monitor mode	Low-latency loopback via AudioRecord + AudioTrack
Layout system	FixedCanvas (FittedBox + 423.5×945.9 dp reference)
Display cutout	LAYOUT_IN_DISPLAY_CUTOUT_MODE_SHORT_EDGES

Platform requirements

OS	Android 7.0 (API 24) or higher
Architecture	arm64-v8a, armeabi-v7a, x86_64
Permissions	RECORD_AUDIO, FOREGROUND_SERVICE, WRITE_EXTERNAL_STORAGE
Approximate APK size	< 30 MB
Storage requirement	~10 MB per minute (WAV) / ~2 MB per minute (AAC)

Implementation

UI layer	Flutter / Dart
Native layer	Kotlin (Android)
Channel name	com.example.lyra_recorder/recorder
Event channel	com.example.lyra_recorder/audio_stream
Audio thread	Dedicated Kotlin thread, real-time priority